Parallel Borůvka Minimum Spanning Tree

Stefano Sacchet
University of Trento
stefano.sacchet@studenti.unitn.it

Luigi Dell'Eva
University of Trento
luigi.delleva@studenti.unitn.it

***Abstract*—This paper presents a comparative study of three parallel implementations of Borůvka's algorithm for computing the Minimum Spanning Tree (MST) of a weighted, undirected graph. The goal is to evaluate the performance and scalability of each approach on high-performance computing infrastructure. The first implementation uses MPI to distribute the workload across multiple processes in a distributed memory environment. The second leverages OpenMP to exploit shared-memory parallelism on multicore systems. The third combines both MPI and OpenMP in a hybrid model, aiming to balance inter-node communication with intra-node concurrency. A custom graph generator was developed to produce large synthetic graphs used for benchmarking. Experimental results demonstrate the strengths and trade-offs of each method in terms of efficiency, scalability, and resource utilization. The MPI and hybrid approaches show strong scalability across nodes, while OpenMP provides a simpler implementation with good performance on single-node.**

I. Introduction

A spanning tree [2] is a tree-like subgraph of a connected, undirected graph that includes all the vertices of the original graph. In simpler terms, it is a subset of the graph edges that forms an acyclic, connected structure where every vertex is reachable and no cycles are present. A graph can have multiple spanning trees. The essential properties of any spanning tree are the following:

- It includes all vertices of the graph.
- It contains exactly $V - 1$ edges, where V is the number of vertices in the graph.
- It is connected (i.e., all vertices are reachable).
- It contains no cycles.
- The total weight (or cost) of a spanning tree is the sum of the weights of its edges.

A minimum spanning tree (MST) [12] is a spanning tree with the smallest possible total weight among all possible spanning trees of a given weighted graph 1. Like spanning trees, a graph can also have multiple MSTs, especially when there are edges with equal weights. Several well-known algorithms can compute an MST efficiently. The following are the most important ones:

a) Kruskal's Algorithm: Kruskal's algorithm [7] begins by sorting all edges of the graph in ascending order based on their weights. It then iteratively examines each edge in this sorted list, adding the edge to the growing minimum spanning tree if and only if it does not create a cycle with the edges already included. To efficiently detect and prevent the formation of cycles, the algorithm typically employs a Disjoint Set Union (DSU), also known as the Union-Find data structure. Due to its conceptual simplicity and computational efficiency, Kruskal's algorithm is widely applied in practical fields such as network design, clustering, and data analysis.

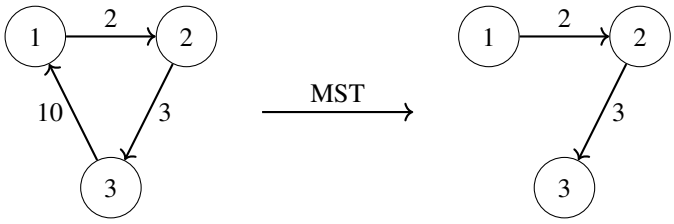

Fig. 1. Minimum Spanning Tree (MST) for Directed Graph [6].

b) Prim's Algorithm: Prim's algorithm [13] is another greedy approach that constructs the minimum spanning tree incrementally. It starts from an arbitrary vertex, which is initially included in the MST. At each step, the algorithm selects the smallest edge that connects a vertex already in the MST to a vertex that has not yet been included. This process is repeated until all vertices are part of the MST. To efficiently select the minimum-weight edge at each iteration, a priority queue (typically implemented as a min-heap) is used. Prim's algorithm is particularly effective for dense graphs and finds practical applications in areas such as image segmentation and routing optimization.

c) Borůvka's Algorithm: Borůvka's algorithm [9], one of the earliest known algorithms to find a minimum spanning tree, also follows a greedy strategy and operates in iterative phases. Initially, each vertex in the graph is treated as an individual component or tree. During each phase, the algorithm identifies the cheapest edge that connects each component to a different component and adds these edges to the MST. Once these edges are added, the connected components are merged accordingly. This process continues until all components are combined into a single spanning tree.

This paper presents an analysis and evaluation of three different parallel implementations of Borvka's algorithm [9]: one that uses **MPI** [11], one that uses **OpenMP** [5], and a hybrid approach.

II. Related Works

This section serves as an introduction to the original Borůvka's algorithm [9]. In particular, it delves into the algorithm serial workflow, also discussing its complexity.

The initial assumptions for the algorithm are that the input graph is connected, weighted, and undirected graph. The algorithm starts by identifying the minimum-weight edge incident to each vertex and adds these edges to form an initial forest. In the following iterations, it continues by finding the minimum-weight edge that connects each existing tree in the forest to a different tree and adds those edges to the forest as well. With each iteration, the number of trees within each connected component is reduced to at most half of its previous count. This process continues for a logarithmic number of steps, after which the resulting set of edges forms the minimum spanning forest. If edges do not have distinct weights, then a consistent tie-breaking rule must be used to ensure that the created graph is indeed a forest, that is, it does not contain cycles.

The serial algorithm described in this part can be found in the corresponding GitHub repository.

a) Complexity: Borůvka's algorithm is shown to take $O(logV)$ iterations of the outer loop until it terminates, and therefore to run in time $O(ElogV)$, where E is the number of edges, and V is the number of vertices in G. In planar graphs, and more generally in families of graphs closed under minor graph operations, it can be made to run in linear time, by removing all but the cheapest edge between each pair of components after each stage of the algorithm [4].

III. Methodology and Implementation

After exploring the serial Borůvka's algorithm, this section delves into the proposed parallel implementations using *MPI* [11] and OpenMP [5], with a focus on the shared structures between the serial and parallel versions, as well as the key changes introduced. Furthermore, this section analyzes the data dependencies of the proposed solution.

A. Common Structures

All approaches replicate the steps of the original serial Borůvka's algorithm; the primary differences lie in how computation is distributed across cores or threads and how data is shared or communicated between them. This is justified by the design of the parallel implementations, which spawn N processes or threads that independently apply the serial logic to distinct sub-portions of the graph. The goal is to maintain correctness while gaining performance through parallelization.

The fundamental data structures are shared across both serial and parallel implementations. The `Edge` structure defines a single edge in the graph and includes the source and destination vertices, along with the weight of the edge. The `Graph` structure represents the entire graph, storing the total number of vertices and edges, and points to an array of `Edge` elements. These representations ensure that all versions of the algorithm operate on a consistent view of the input graph.

Algorithm 1 Borůvka's MST [15]

Input: A weighted undirected graph $G = (V, E)$
Output: F, a minimum spanning forest of G.

Initialize forest $F \leftarrow (V, E')$ where $E' \leftarrow \emptyset$
$completed \leftarrow$ false
while not $completed$ **do**
    for all edges $uv \in E$ where u and v are in different components **do**
        $wx \leftarrow$ cheapest edge for the component of u
        if IS_PREFERRED_OVER(uv, wx) **then**
            Set uv as the cheapest edge for the component of u
        end if
        $yz \leftarrow$ cheapest edge for the component of v
        if IS_PREFERRED_OVER(uv, yz) **then**
            Set uv as the cheapest edge for the component of v
        end if
    end for
    if all components have cheapest edge set to `None` **then**
        $completed \leftarrow$ true
    else
        $completed \leftarrow$ false
        for all components with cheapest edge $\neq$ `None` **do**
            Add its cheapest edge to E'
        end for
    end if
end while

function IS_PREFERRED_OVER($edge_1$, $edge_2$)
    return $edge_2$ is `None` **or** $weight(edge_1) < weight(edge_2)$ **or** $(weight(edge_1) = weight(edge_2)$ **and** TIE_BREAKING_RULE($edge_1$, $edge_2$))
end function

function TIE_BREAKING_RULE($edge_1$, $edge_2$)
    return true if and only if $edge_1$ is preferred over $edge_2$ in case of a tie
end function

Listing 1 `Graph` and `Edge` data structures definition

```
struct Edge {
  edge_t src, dest;
  edge_weight_t weight;
};

struct Graph {
  graph_size_t V, E;
  Edge_t *edges;
};
```

The algorithm also relies on two essential functions that support the **Disjoint Set Union** (DSU) data structure: `Find` 2 and `UnionSets` 3. The Find function, implemented with

path compression, determines the representative element of the subset to which a given vertex belongs. This operation flattens the tree structure of subsets over time, significantly improving lookup efficiency. The UnionSets function merges two disjoint sets using the union-by-rank technique, attaching the tree with lower rank to the one with higher rank. If both have the same rank, one becomes the parent and its rank is increased. These two functions are critical for preventing cycles and efficiently managing connected components during the construction of the minimum spanning tree.

In the parallel implementations, these structures and functions remain largely unchanged. However, their usage is adapted to accommodate parallel data access and communication between threads or processes. This ensures that even when operating concurrently, each worker contributes correctly to the global solution.

Algorithm 2 FIND with Path Compression

```
1: function FIND(subsets, i)
2:     if subsets[i].parent ≠ i then
3:         subsets[i].parent ← FIND(subsets, subsets[i].parent)
4:     end if
5:     return subsets[i].parent
6: end function
```

Algorithm 3 UNIONSETS with Union by Rank

```
1: function UNIONSETS(subsets, x, y)
2:     rootX ← FIND(subsets, x)
3:     rootY ← FIND(subsets, y)
4:     if rootX ≠ rootY then
5:         if subsets[rootX].rank < subsets[rootY].rank then
6:             subsets[rootX].parent ← rootY
7:         else if subsets[rootX].rank > subsets[rootY].rank then
8:             subsets[rootY].parent ← rootX
9:         else
10:            subsets[rootY].parent ← rootX
11:            subsets[rootX].rank ← subsets[rootX].rank + 1
12:        end if
13:    end if
14: end function
```

B. MPI

As previously discussed, Borůvka's algorithm begins by treating each node as an individual component (or tree) and repeatedly merges these components by selecting the minimum weight outgoing edge for each. This merging continues iteratively until all components are unified into a single spanning tree. Due to its inherently repetitive and component-based structure, the algorithm lends itself well to parallelization.

The first proposed approach uses *MPI* [11], the Message Passing Interface, a standard for parallel programming on distributed memory systems. MPI allows processes running on different nodes to communicate through explicit message passing, using operations like `MPI_Send`, `MPI_Recv`, and collective routines such as `MPI_Bcast`. It follows the **SPMD** (Single Program Multiple Data) model, where each process executes the same program but works on different parts of the data.

A straightforward approach to parallelize Borůvka's algorithm using MPI involves partitioning the graph among multiple processes. Each process is assigned a subgraph and is responsible for performing the local computation of finding the cheapest outgoing edges for the components within its partition. These local results, typically sets of candidate edges for merging, are then communicated to a master process. The master process collects the candidate edges from all workers, resolves global conflicts (such as duplicate or cyclic edges), and applies union operations to merge the global forest. After each iteration, the updated component information is redistributed as needed, and the process continues until a single connected component (the MST) remains. This parallel strategy builds on a baseline MPI implementation initially sourced [10]. Although the original version offered a functional foundation, several enhancements were introduced to improve both robustness and performance. These modifications include more efficient memory usage, finer-grained communication logic, and better load balancing through `MPI_ScatterV`. Furthermore, we ensured correctness through careful management of component states and introduced mechanisms to reduce unnecessary synchronization and communication overhead.

Delving into the implementation details, the solution begins with the master process parsing the graph and loading it into memory using the `Graph` structure (see *Listing* 1). It is important to note that the graph parsing time is excluded from the benchmarks measured time, as the focus lies only on the computation time of the MST construction. Once the graph has been parsed, the master process distributes subsets of the edge list to the worker processes. To ensure load balancing, that is, assigning each worker an equal number of edges, the implementation uses `MPI_ScatterV`. This function is an extended version of `MPI_Scatter`, which allows each receiving process to get a different number of elements and supports flexible placement of those elements in the root send buffer. It is a collective communication operation, meaning that all processes within the communicator must call it simultaneously to ensure a correct execution [14].

After each worker received its subgraph, the core part of the parallel Borůvka's algorithm can start. It consists of multiple iterations, each aimed at reducing the number of connected components by selecting and merging the cheapest edges that connect distinct components. At the start of each iteration, the algorithm resets the cheapest array, which stores the lightest edge found so far for each component. It then traverses the local edge partition (`edges_part`) and, for each edge, determines whether it connects two different components using

the `find` 2 function (which implements path compression). If the two endpoints belong to different components, the algorithm checks whether the current edge has a lower weight than the cheapest edge currently recorded for either of the two components. If so, it updates the entry in the cheapest array accordingly. This ensures that, at the end of the traversal, each process has locally identified the minimum-weight outgoing edge for each component it is responsible for. To construct a globally consistent MST, these local cheapest edges must be merged across all processes. This is done through a parallel reduction operation. The algorithm begins a *binary-tree style reduction*, where processes with even-numbered ranks receive cheapest arrays from neighbouring higher-ranked processes (i.e., ranks offset by powers of two) and merge them by comparing corresponding edge weights. This is achieved by iteratively doubling the size of the reduction step ($step* = 2$), ensuring that all processes contribute to the final global result while maintaining logarithmic communication complexity. Only the process with rank zero ends up with the fully reduced global cheapest array, which is then broadcast to all worker processes. This array contains the minimum outgoing edge for each component throughout the entire graph. Each process subsequently performs the union operations on these edges to merge the corresponding components. However, only the master process is responsible for updating the MST by adding the selected edges. Despite this, all workers perform the same union operations locally, ensuring that the component information remains consistent across all processes. The whole communication logic can be found at Fig: 2. This approach is adopted to avoid the overhead of communicating updated component data, as communication costs are typically higher than the cost of performing these local computations. This process repeats until the MST is complete, i.e., until the number of edges in the MST equals $n\_vertices - 1$. The algorithm described in this part can be found in the corresponding GitHub repository.

Additionally, the code includes several *assert* statements to detect potential overflows and ensure the correctness of variable values. It is important to note that these checks do not affect the algorithm runtime performance in production, as they are included only in the *debug* compilation mode. When the code is compiled in *release* mode, these assertions are omitted.

C. OpenMP

The second parallel approach explored was based on *OpenMP* [5]. Unlike MPI, which relies on message passing between multiple independent processes, OpenMP is a shared memory model that uses threads running within a single process. This fundamental distinction leads to significant differences in the implementation, although the overall parallelization strategy, targeting the most computationally intensive parts of Borůvka's algorithm, remains logically similar. While the MPI based implementation requires explicit data distribution, communication, and synchronization between processes, the OpenMP version works within a shared memory space, allowing threads to access common data structures directly. As a result, OpenMP simplifies memory management and eliminates the need for explicit data communication, but requires careful handling of concurrent access to shared variables to avoid race conditions. In contrast, MPI implementations are more scalable and offer finer control over communication.

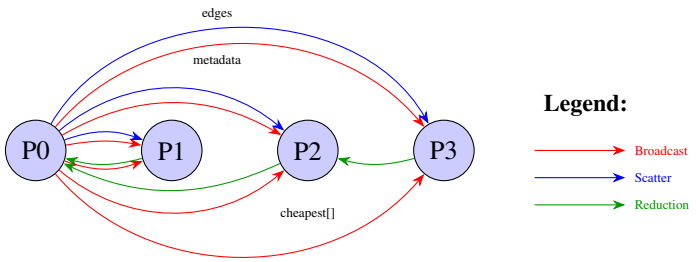

Fig. 2. MPI Communication Pattern in Parallel Borůvka MST Algorithm. The algorithm uses three main communication phases: (1) Initial broadcast of graph metadata and scattering of edge partitions, (2) Tree-based reduction to find globally cheapest edges per component in each Boruvka iteration, and (3) Broadcasting of final results back to all processes.

The core structure of the algorithm remains consistent with the MPI version, involving repeated rounds of selecting the minimum weight outgoing edge for each component and performing union operations to merge the components. However, parallelism is achieved differently. At the beginning of the algorithm, the `subsets` array (used for union-find operations) and the `cheapest` array (used to store the minimum outgoing edge for each component) are initialized in parallel using OpenMP `simd` (Single Instruction Multiple Data) directive. Not having any data dependency enables efficient memory initialization across threads with minimal overhead. To avoid data races during the computation of the cheapest edges, each thread is allocated a private buffer in a contiguous region of a preallocated array. These buffers store the best local edges independently, eliminating the need for synchronization during this phase. The main computation loop proceeds by having each thread iterate over a portion of the graph's edge list, identifying the cheapest outgoing edges for the components it encounters. These local results are then merged into the global cheapest array. This merge phase is performed in parallel using nested loops: the outer loop iterates over vertices, and the inner loop compares the best edge found by each thread for the current vertex, keeping the minimum. Once all the cheapest edges are identified, a candidate list is assembled by collecting only the valid edges (i.e., those that connect distinct components). Due to complicated race conditions, the actual merging of components and updating of the MST are performed sequentially. For each edge in the candidate list, the algorithm checks whether the endpoints belong to different components. If so, it adds the edge to the MST and merges the components using the union operation 3. This phase guarantees correctness and avoids data corruption due to concurrent modifications. The algorithm terminates when the number of

edges in the MST equals to the number of vertices minus one. This OpenMP based approach benefits from efficient intra-node parallelism, reduced memory overhead, and a simpler implementation compared to MPI.

Finally, all the for loops in the algorithm are parallelized using `omp` directives, except for the merging phase, which remains sequential due to the intricate race conditions in the union-find data structure. Trivial loops that do not involve data dependencies, such as array initialization, are parallelized using the `simd` directive. This enables fine-grained data-level parallelism and can significantly accelerate simple loops that operate on contiguous memory with no inter-iteration dependencies. In contrast, for the non-trivial loops involving dynamic workloads, extensive benchmarking led to the decision to use the `schedule(auto)` directive. This allows OpenMP runtime to choose the most suitable scheduling strategy depending on the system and workload characteristics, resulting in better performance and load balancing.

The algorithm described in this section can be found in the corresponding GitHub repository.

a) Data Dependencies: Data dependency analysis in the context of parallel programming is used to detect and manage relationships between different parts of the code that influence execution order. Its main goal is to ensure that parallel tasks operate without conflicts, avoiding race conditions and maintaining data correctness. This analysis helps identify which operations can be safely executed concurrently and which require sequential execution. By doing so, it enables both safe and efficient parallelization, improving performance while preserving the correctness of the program. The analysis consists of three phases:

1) **Detection**: identifies all potential data dependencies in the code (e.g., true dependencies, anti-dependencies, output dependencies).
2) **Classification**: categorizes each dependency to determine its nature and impact (e.g., loop-carried, loop-independent).
3) **Removal**: focusses on transforming the code to eliminate or minimize harmful dependencies where possible.

This section discusses the results of the data dependency analysis, with a focus on the loop responsible for finding the cheapest outgoing edge for each component, shown in Fig. 3. The identified data dependencies for this code segment are summarized in Table I. The analysis started with the detection of potential dependencies by examining the following variables:

- `current_edge`
- `set1`
- `set2`
- `cheapest`

For variables `current_edge`, `set1`, and `set2`, no data dependencies are detected, as they are computed independently in each loop iteration. This means that the value of these variables at iteration i does not depend on the value at iteration $i + 1$, making them safe for parallel execution. However, the variable `cheapest` introduces potential data hazards. Since multiple iterations can concurrently access and modify elements of the `cheapest` array at indices corresponding to `set1` and `set2`, there is the possibility of *flow* (write-read), *anti* (read-write) and *output* (write-write) dependencies. Specifically, two different iterations might attempt to update the same index `cheapest[v]` simultaneously if different edges are incident on the same component v, leading to a race condition. This dependencies prevents the loop from being trivially parallelized. As a result, to safely parallelize this loop, a mechanism to eliminate or avoid concurrent writes is necessary.

To address this issue, the removal phase involved redesigning the loop to use thread-local buffers. Each thread maintains its own local copy of the `cheapest` array (e.g., `my_cheapest`), avoiding conflicts. Once all threads complete their local computations, a reduction step merges the local arrays into the global `cheapest` array using deterministic conflict resolution (e.g., choosing the edge with the lowest weight). This approach preserves correctness and enables efficient parallel execution.

The modified version of the loop, shown in Fig. 4, illustrates the adopted parallelization strategy. For clarity and brevity, the initialization of arrays has been omitted from the figure.

```
1  for (uint64_t i = 0; i < E; i++) {
2    Edge_t current_edge = edge[i];
3    uint64_t set1 = find(subsets,
     ↪  current_edge.src);
4    uint64_t set2 = find(subsets,
     ↪  current_edge.dest);
5
6    if (set1 != set2) {
7      if (cheapest[set1].weight == -1 ||
       ↪  cheapest[set1].weight >
       ↪  current_edge.weight) {
8        cheapest[set1] = current_edge;
9      }
10     if (cheapest[set2].weight == -1 ||
       ↪  cheapest[set2].weight >
       ↪  current_edge.weight) {
11       cheapest[set2] = current_edge;
12     }
13   }
14 }
```

Fig. 3. Serial cheapest edge finding loop.

Memory Location	Earlier Statement			Later Statement			Loop Carried?	Kind of Dataflow
	Line	Iter.	Access	Line	Iter.	Access		
`cheapest[set1]`	7	i	read	8	i	write	✗	anti
`cheapest[set1]`	8	i	write	7	$j > i$	read	✓	flow
`cheapest[set1]`	8	i	write	8	$j > i$	write	✓	output
`cheapest[set2]`	10	i	read	11	i	write	✗	anti
`cheapest[set2]`	11	i	write	10	$j > i$	read	✓	flow
`cheapest[set2]`	11	i	write	11	$j > i$	write	✓	output

TABLE I
DATA DEPENDENCIES IN THE CHEAPEST EDGE FINDING LOOP.

D. Hybrid

The third and final approach explored combines both *MPI* and *OpenMP*, leveraging their complementary strengths in a

```
#pragma omp for schedule(auto)
for (uint64_t i = 0; i < n_edges; i++) {
    Edge_t current_edge = edges[i];
    uint64_t set1 = find(subsets, current_edge.src);
    uint64_t set2 = find(subsets, current_edge.dest);
    if (set1 != set2) {
      if (my_cheapest[set1].weight == -1 ||
          my_cheapest[set1].weight > current_edge.weight) {
        my_cheapest[set1] = current_edge;
      }
      if (my_cheapest[set2].weight == -1 ||
          my_cheapest[set2].weight > current_edge.weight) {
        my_cheapest[set2] = current_edge;
      }
    }
}
#pragma omp for schedule(auto)
for (uint64_t v = 0; v < n_vertices; v++) {
    for (int t = 0; t < num_threads; t++) {
        Edge_t *thread_cheapest = &local_cheapest[t * n_vertices];
        if (thread_cheapest[v].weight != -1 &&
            (cheapest[v].weight == -1 || cheapest[v].weight > thread_cheapest[v].weight)) {
            cheapest[v] = thread_cheapest[v];
        }
    }
}
```

Fig. 4. Parallel cheapest edge finding loop.

hybrid parallelization model. In this approach, *MPI* is used for inter-node communication, distributing different parts of the graph across multiple processes, while *OpenMP* is employed within each process to parallelize computations across multiple threads on the same node.

The hybrid model follows a structure similar to the pure *MPI* version, where each process handles a partition of the graph and computes local candidates for the minimum outgoing edge of each component. However, within each process, *OpenMP* is used to parallelize the most computationally intensive tasks such as edge scanning, local minimum edge selection, and the reduction of thread-local results. This strategy reduces the total number of MPI processes needed, thereby decreasing the volume and frequency of inter-process communication, which is typically more expensive than intra-process synchronization. Moreover, it increases scalability and allows better exploitation of modern multi-core, multi-node architectures. In summary, the hybrid approach aims to balance communication overhead with computational efficiency by combining coarse-grained parallelism (via MPI) with fine-grained parallelism (via OpenMP), achieving both performance gains and improved resource utilization.

The algorithm described in this section can be found in the corresponding Github repository.

E. Compilation and Run

The project uses a Makefile that wraps around the underlying CMake configuration responsible for building the project. It supports compilation in both *debug* and *release* modes: the debug mode disables compiler optimizations (using `-O0`), while the release mode enables maximum optimization (using `-O3`). Additionally, the Makefile provides several helper targets for tasks such as submitting, monitoring, or cancelling jobs on the computing cluster. The build process produces six executables. Three of these correspond to the previously discussed parallelization strategies. The remaining executables are: *graph_generator*, which creates graphs with an arbitrary number of nodes and edges (further discussed in Section IV-B); *serial_mst*, which contains the baseline sequential implementation of the algorithm; and *test_all*, which runs automated tests to verify the correctness of the solutions. It is also possible to compile a single executable by specifying the desired target using `RUN_TYPE=X` when invoking `make`, where `X` corresponds to the name of the executable. The decision to separate the program into multiple executables was made to ensure that each implementation remains independent and as lightweight as possible, including only the components necessary for its execution.

However, to run the code on the cluster, it is necessary to use *PBS* directives to submit jobs to the compute nodes. To facilitate this, the GitHub repository includes several shell scripts that automate the entire process, including compiling the code and creating the appropriate output directories. A snippet of an example script is shown in Fig. 5. The script expects four input arguments:

- `num_processes`: the number of MPI processes to spawn.
- `input_file`: the path to the input graph file.
- `cpus`: the number of nodes to allocate.
- `placement`: the process placement strategy (*pack* or *scatter*).

These parameters allow for flexible configuration of job submissions for different test scenarios.

IV. Experimental Evaluation

This section presents the experimental results obtained from the parallel algorithm implementation. The main objective is to evaluate the performance and efficiency of the proposed algorithm. By examining various metrics and comparing the results with those obtained with the serial Borůvka algorithm, this part aims to provide insights into the algorithm strengths, weaknesses and overall effectiveness.

```
#!/bin/bash
#PBS -l
↪ select=${cpus}:ncpus=$num_proc:mem=64gb
#PBS -l place=${placement}:excl
#PBS -l walltime=00:20:00
#PBS -q short_cpuQ
#PBS -N mpi_mst${num_proc}_${placement}
#PBS -o <stdout_log_path>
#PBS -e <stderr_log_path>
module load mpich-3.2
mpirun.actual -n $num_proc
↪ ${PWD}/build/bin/mpi_mst
↪ "$input_file"
```

Fig. 5. PBS script used to submit MPI jobs to the cluster. `<stdout_log_path>` and `<stderr_log_path>` are placeholders for the real output path.

A. HPC Cluster

To evaluate the parallelization strategies on a large number of cores, the High Performance Computing (HPC) cluster at the University of Trento was used. The cluster is managed by the Altair PBS Professional workload manager and runs on CentOS 7 Linux across its compute nodes. The HPC infrastructure comprises a total of 126 nodes and offers 6092 CPU cores and 37,376 CUDA cores, with an aggregate RAM capacity of 53 TB. The nodes are interconnected through a 10 Gb/s Ethernet network, with some nodes also featuring high-speed interconnects such as InfiniBand at 40 Gb/s and Omni-Path at 100 Gb/s.

B. Benchmark Datasets

In order to evaluate the performance of the proposed parallel solutions, a custom graph generator was implemented. This generator is compiled as a stand-alone executable and is designed to create synthetic graphs with a specified number of vertices and edges. It accepts two command-line arguments: the number of vertices and the number of edges to generate. The executable can be invoked as follows:

```
./build/bin/graph_generator <num_vertices>
↪ <num_edges>
```

The generation process begins by ensuring that the resulting graph is connected. This is achieved by constructing a spanning tree, which connects all vertices with exactly $V-1$ edges (where V is the number of vertices), using randomly selected edges. Each of these edges is assigned a random weight from 1 to 1000. This step guarantees that there exists at least one valid spanning tree in the final graph. Once the initial connectivity is established, the generator proceeds to add additional edges at random until the desired number of edges is reached. These edges are also assigned random weights in the same previous range. The final graph does not follow any specific real-world topology, but it serves well for benchmarking purposes due to its controlled size and randomized structure. All performance benchmarks in this work were carried out using graphs generated by this tool. This decision was motivated by the absence of suitable publicly available graph datasets that meet the requirements for evaluating parallel algorithms on a high number of processing cores. Most available datasets are either too small or too sparse to fully exploit the capabilities of parallel processing environments. Synthetic graphs, on the contrary, offer full control over graph size, density, and structure, enabling systematic and reproducible evaluation across a wide range of scenarios.

To determine the appropriate graph sizes for benchmarking the proposed solutions, some considerations were taken into account. The maximum number of edges in an undirected graph with V vertices is given by $\frac{V(V-1)}{2}$. Based on this, a range of sparse and dense graphs of varying sizes classified as small, medium, and large was selected for evaluation. The density of an undirected graph is defined as:

$$Density = \frac{2E}{V(V-1)}$$

where E is the number of edges and V is the number of vertices [3]. Table II provides an overview of the datasets used in the experiments.

Graph Type	Name	Vertices (#)	Edges (#)	Density (%)
Small Dense	*10k45m*	10K	45M	90%
Medium Sparse	*80k65m*	80k	65M	2%
Medium Dense	*80k2b*	80K	2B	62%
Large Sparse	*500k1b*	500K	1B	0.8%

TABLE II
DENSITY FOR BENCHMARK GRAPHS.

To facilitate the evaluation of benchmark results, a custom Python based plotting tool was developed using *Matplotlib* [8]. In addition to generating plots, the tool also automates job submission to the HPC cluster by invoking the appropriate shell scripts. All runs are organized into subfolders (e.g., *mpi*, *omp*, *pack*, *scatter*) to ensure a clear and consistent directory structure. The plotter generates a variety of visualizations, including speedup and efficiency curves, as well as comparisons across different numbers of nodes.

C. Metrics Definition

In order to better understand and evaluate the performance of the proposed solutions, it is necessary to define the metrics used for a more detailed performance analysis. All metrics are computed based on the average execution time recorded over multiple runs for each solution. The serial baseline is obtained by running the original algorithm without any framework overhead and the maximum optimization. The performance metrics considered are:

- **Speedup**: $S = \frac{T_s}{T_p}$ the speedup is defined as the ratio of the serial runtime T_S of the best sequential algorithm to solve a problem to the time taken by the parallel algorithm T_p to solve the same problem on p processors.
- Efficiency: $\frac{S}{p}$ the efficiency is defined as the ratio of speedup S to the number of processors p. Efficiency measures the fraction of time for which a processor is usefully utilized.
- **Scalability**: the scalability of the algorithm is evaluated in two distinct ways:
  - *Strong scalability*: This assesses how efficiency changes when the problem size remains fixed while

the number of processing cores increases. An algorithm is considered strongly scalable if the efficiency remains constant under these conditions.

– *Weak scalability*: this assesses how efficiency varies when both the problem size and the number of cores increase proportionally, maintaining a constant workload per core. If efficiency remains stable in this scenario, the algorithm is considered weakly scalable.

To evaluate the performance of all solutions, the **execution time**, **speedup**, and **efficiency** across varying numbers of processes and node configurations (1, 2, and 4 nodes) was analysed. The analysis compares two communication strategies (Pack and Scatter) and is based on average execution times collected over multiple runs.

It is important to note that, for reasons of space and length, only the results for the largest dataset are presented here, as it best illustrates the benefits of parallelization. However, the complete set of benchmark results conducted on four different graphs, as described in Table II, with a total of sixty plots can be accessed in the shared Google Drive folder.

D. MPI Results

Figures 6, 7 show the execution time as a function of the number of processes. As expected, increasing the number of processes leads to a reduction in the total execution time for all configurations. This clearly demonstrates the potential for parallelism in such algorithms. The Pack strategy consistently outperforms Scatter, especially as the number of processes increases. Among the tested CPU configurations, the 1 CPU version of Pack scales the best up to 64 processes, closely followed by the 2 CPU version. The Scatter approach, particularly with 4 CPUs, shows less favourable scaling behaviour, exhibiting a noticeable performance bottleneck as the number of processes grows. This outcome was anticipated, as the 1 node Pack strategy minimizes inter-process communication, which becomes increasingly costly at higher process counts. In contrast, the Scatter strategy suffers from greater communication overhead, which hinders its scalability and leads to lower performance at larger scales. However, the benefits of adding more cores are subject to diminishing returns. In fact, while the initial increase seems always to result in a significant speedup, further increases yield progressively smaller improvements. This may indicate an optimal range for core usage, probably based on dataset size, where the balance between performance gain and resource allocation is the most efficient.

Speedup results are shown in Figures 8, 9. The Pack implementation demonstrates near-linear speedup for lower core counts and continues to scale reasonably well up to 64 processes. In contrast, Scatter achieves more modest gains and exhibits diminishing returns beyond 16 processes. This indicates that Pack is more effective at exploiting parallelism within the available hardware. The best speedup is obtained with the 1 CPU - Pack configuration, reaching a speedup of nearly 10 with 64 processes.

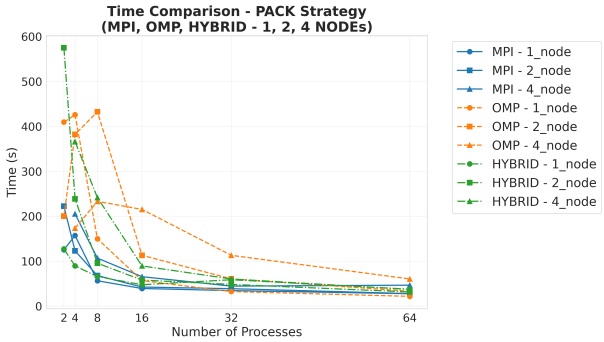

Fig. 6. Execution time comparison using the *Pack* strategy for MPI, OpenMP (OMP), and Hybrid (MPI+OpenMP) implementations across 1, 2, and 4 nodes. The graph shows the performance trend as the number of processes increases from 2 to 64.

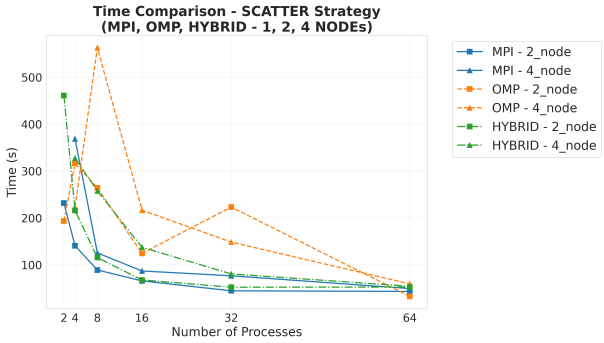

Fig. 7. Execution time comparison using the *Scatter* strategy for MPI, OpenMP (OMP), and Hybrid (MPI+OpenMP) implementations across 1, 2, and 4 nodes. The graph shows the performance trend as the number of processes increases from 2 to 64.

Efficiency trends (Figures 10, 11) reveal the rate at which additional processes contribute to actual performance improvements. Efficiency naturally decreases as the number of processes increases due to communication and synchronization overheads. However, as mentioned earlier, the Pack configuration maintains higher efficiency than Scatter across all test cases. The 1 CPU - Pack implementation starts with perfect efficiency at low process counts (likely due to reduced overhead) and degrades as the process count rises. Conversely, Scatter efficiency drops quickly and remains low, especially with 4 CPUs.

E. OpenMP Results

The OpenMP implemented solution, as shown in Figures 6, 7, 8, 9, 10, 11, exhibits similar trends in all metrics for time, speedup, and efficiency. However, it consistently shows worse performance compared to the MPI implementation. Although OpenMP provides a simpler and more concise parallel programming model (particularly effective for shared-memory architectures), this simplicity comes at a cost. One significant drawback is the potential for race conditions, which can arise if shared data is not properly protected or if thread synchronization is not correctly handled. These issues can lead to incorrect results or degraded performance due to the overhead

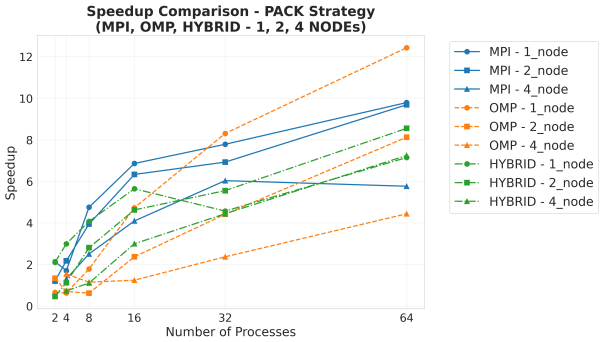

Fig. 8. Speedup comparison using the *Pack* strategy for MPI, OpenMP (OMP), and Hybrid (MPI+OpenMP) implementations across 1, 2, and 4 nodes. The plot shows how each configuration scales with increasing numbers of processes.

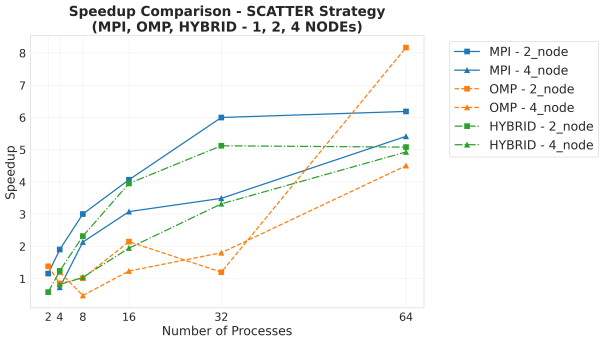

Fig. 9. Speedup comparison using the *Scatter* strategy for MPI, OpenMP (OMP), and Hybrid (MPI+OpenMP) implementations across 1, 2, and 4 nodes. The plot shows how each configuration scales with increasing numbers of processes.

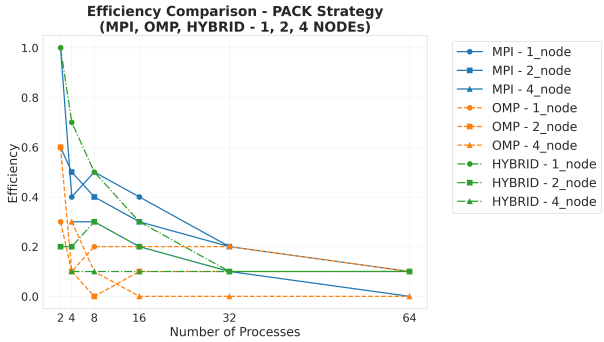

Fig. 10. Efficiency comparison under the *Pack* strategy for MPI, OpenMP (OMP), and Hybrid (MPI+OpenMP) implementations on 1, 2, and 4 nodes. Efficiency is calculated as speedup divided by the number of processes.

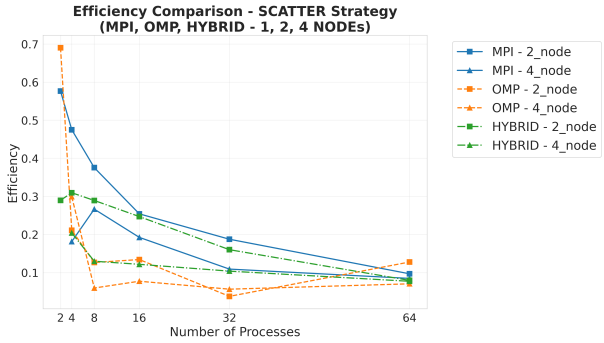

Fig. 11. Efficiency comparison under the *Scatter* strategy for MPI, OpenMP (OMP), and Hybrid (MPI+OpenMP) implementations on 1, 2, and 4 nodes. Efficiency is calculated as speedup divided by the number of processes.

of synchronization mechanisms. Moreover, OpenMP scalability is inherently limited by the shared memory model, as the number of threads increases, contention for shared resources (e.g., caches and memory bandwidth) becomes a bottleneck, reducing overall efficiency. In addition, some performance spikes are observed in the results. These anomalies can often be attributed to transient delays in the cluster infrastructure or the scheduling of computations on less performant CPUs. Such inconsistencies are particularly relevant in shared computing environments, where background load, thermal throttling, or resource contention can unpredictably affect execution time and parallel efficiency.

F. Hybrid Results

The hybrid solution, which combines MPI and OpenMP, exhibits performance trends that are largely comparable to the pure MPI implementation. In some scenarios, the hybrid approach achieves slightly better performance due to the reduced overhead of intra-node communication, leveraging OpenMP threads within each node while using MPI for inter-node communication. However, in other cases, the hybrid approach performs slightly worse than pure MPI. This can be attributed to increased complexity, which may introduce overhead or inefficiencies, particularly if the workload is not well-balanced between threads and processes. Additionally, tuning the hybrid configuration (e.g., the number of MPI processes per node and the number of threads per process) is critical and can significantly impact performance. In general, the hybrid implementation offers a flexible compromise between the simplicity and locality of OpenMP and the scalability of MPI. Altough it does not consistently outperform the pure MPI version, it remains a viable and often efficient alternative.

G. Scalability

From Figures 10 and 11, it is also possible to evaluate the strong scalability of all parallel algorithms. As noted above, efficiency decreases as the number of processes increases. Therefore, none of the evaluated approaches demonstrates strong scalability. Figure 12 evaluates weak scalability. The graph reveals a pseudo-linear decline in efficiency, indicating that the parallel algorithms do not exhibit weak scalability either. The observed decrease in efficiency in the weak scalability graph suggests that the parallel overhead increases faster than the performance gains from distributing the workload. This can be due to:

- Limited Parallel Fraction: According to Amdahl's Law [1], a non-parallelized portion of the code becomes a bottleneck, limiting scalability (e.g., non-parallelized union set).

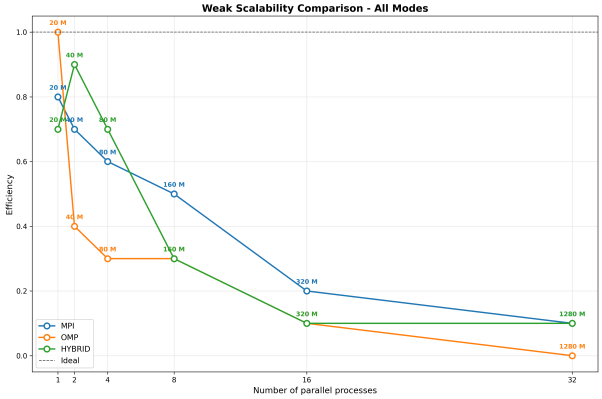

Fig. 12. Weak Scalability.

- Communication Overhead: as more nodes and processes are used, inter-process communication grows, especially in MPI and hybrid approaches.
- Resource Contention: With more threads or processes per node, resources like memory bandwidth or cache may become saturated.

The main problem limiting scalability is the lack of parallelization in the components merge phase. As a result, even when the problem size increases proportionally with the number of processes, the overhead grows faster than the computational gains. This leads to a drop in efficiency and demonstrates that the algorithms do not achieve weak scalability.

V. Conclusion

This paper thoroughly presents, a comparative performance analysis of parallel implementations using MPI, OpenMP, and a hybrid MPI+OpenMP approach. The goal was to evaluate the scalability and efficiency of each solution when applied to the minimum spanning tree computational problem on a multi-core and multi-node architecture. The experimental results, based on execution time, speedup and efficiency metrics, show that MPI implementation delivers the best overall performance and scalability, particularly for larger problem sizes and higher process counts. MPI explicit communication model enables better utilization of distributed resources. The OpenMP implementation, while easier to implement and effective for shared-memory systems, is hindered from race conditions, synchronization overhead, and limited scalability due to resource contention. It consistently performs worse than MPI, although it remains a viable option for smaller systems or when simplicity and development speed are prioritized. The hybrid approach demonstrates performance close to that of MPI alone. In some cases, it outperforms MPI, in others it underperforms. The hybrid model offers a flexible compromise and can be particularly effective when appropriately tuned for the underlying hardware architecture. Finally, the study confirms that the choice of the most suitable parallelization strategy is highly dependent on the target system architecture and the graph characteristics. While MPI remains the most robust option for high-performance distributed systems, the hybrid approach offers promising advantages in specific scenarios, and OpenMP serves as a practical entry point for simpler shared-memory environments.

Finally, testing revealed significant performance fluctuations between identically configured runs. Initially, this variability was attributed to resource contention caused by other concurrent jobs on the same node. However, even after enabling the `excl` flag, which requests exclusive access to the entire CPU for the job, the performance inconsistencies persisted, although with reduced variance. This behaviour is likely due to the presence of older or less performant machines within the cluster, which introduce variability in execution times despite identical job configurations.

VI. Future Work

This study opens several directions for future exploration and optimization. The parallel version of Borůvka's algorithm appears to be a promising approach for computing minimum spanning trees on large graphs. However, there are several areas for improvement. A key enhancement would be the parallelization of the component merging phase, which currently acts as a bottleneck and limits overall scalability. Another area of interest is the tuning and refinement of the hybrid MPI+OpenMP configuration, specifically identifying the optimal ratio of MPI processes to OpenMP threads per node. This tuning is particularly important for modern multicore architectures, where efficient resource utilization can have a significant impact on performance. Furthermore, the OpenMP solution can be further optimized by addressing its current synchronization overhead and potential race conditions. This may involve redesigning critical sections, improving loop scheduling strategies, and making better use of OpenMP features. These optimizations would be particularly valuable in shared memory systems with many cores, where even small inefficiencies can scale poorly. Finally, better efficiency and scalability could potentially be achieved by testing the implementations on even larger graphs. Increasing the problem size would provide a better match for high-performance computing systems, where communication overhead tends to be reduced over more substantial workloads.

References

[1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, AFIPS '67 (Spring), page 483–485, New York, NY, USA, 1967. Association for Computing Machinery.

[2] Norman Biggs. *Spanning trees and associated structures*, page 31–37. Cambridge Mathematical Library. Cambridge University Press, 1974.

[3] Subham Datta. https://www.baeldung.com/cs/graph-density.

[4] David Eppstein. Spanning trees and spanners. 1996.

[5] MPI Forum. https://www.mpi-forum.org/docs/mpi-3.0/mpi30-report.pdf.

[6] GeeksforGeeks. https://www.geeksforgeeks.org/what-is-minimum-spanning-tree-mst/.

[7] Joseph B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. In *Proceedings of the American Mathematical Society*, 1956.

[8] matplotlib.org. https://matplotlib.org.

[9] Jaroslav Nešetřil, Eva Milková, and Helena Nešetřilová. Otakar borůvka on minimum spanning tree problem translation of both the 1926 papers, comments, history. *Discrete Mathematics*, 233(1):3–36, 2001. Czech and Slovak 2.
[10] Nikitwani07. https://github.com/nikitawani07/MST-Parallel.
[11] OpenMP. https://www.openmp.org.
[12] Seth Pettie. *Minimum Spanning Trees*, pages 1–5. Springer US, Boston, MA, 2008.
[13] R. C. Prim. Shortest connection networks and some generalizations. *The Bell System Technical Journal*, 36(6):1389–1401, 1957.
[14] Rookiehpc. https://rookiehpc.org/mpi/docs/mpi_scatterv/index.html.
[15] Wikipedia. https://en.wikipedia.org/wiki/Borvka%27s_algorithm.